Vacation-Rental Marketplace

High-level production-scale architecture | Frontend, Backend, Storage/Database, Search, Deployment

1 FRONTEND (clients)

Guest Web App
Next.js / React, SSR for SEO, search, listing pages, checkout

Host Dashboard
React SPA: listings, calendar, pricing, bookings, payouts

Mobile Apps
iOS / Android (React Native), push notifications

Admin Console
Support, trust & safety, moderation, finance tools

HTTPS

2 EDGE & ACCESS

DNS (Route 53)
Geo / latency routing

CDN (CloudFront)
Static assets, listing images

WAF + DDoS Shield
Bot & abuse protection

API Gateway + Load Balancer
Auth, rate limit, routing

routed requests

3 BACKEND (stateless microservices on Kubernetes)

User & Auth
Sign-up, login (OAuth/JWT), profiles, KYC status

Listing Service
Properties, amenities, rules, photo metadata

Availability & Pricing
Calendar, dynamic rates, taxes, fees

Booking Service
Reservations, holds, cancellations, state machine

Payments & Payouts
Charge, escrow, host payouts, refunds

Search API
Queries, filters, geo, ranking, suggestions

Messaging
Guest-host chat, real-time via WebSocket

Reviews & Ratings
Two-sided reviews, score aggregation

Notification Service
Email, SMS, push templates and delivery

Media Service
Image upload, resize, virus scan, moderation

Each service: containerized, autoscaled (HPA), owns its data, communicates via REST/gRPC (sync) and Kafka events (async).

APIs

EXTERNAL SERVICES

Payment Gateway
Stripe / Adyen

Messaging Providers
SES, Twilio, FCM / APNs

Maps & Geocoding
Google Maps / Mapbox

Identity Verification
Onfido / Persona

Channel Sync
iCal, Airbnb, Booking.com

Analytics
Segment, Snowflake / BI

publish / subscribe

Event Bus - Apache Kafka (MSK)
booking.created | payment.captured | listing.updated | message.sent | review.posted (async, decoupled, replayable)

read / write

change events

4 STORAGE & DATABASE

PostgreSQL
Primary + read replicas, multi-AZ. Users, listings, bookings, payments (ACID)

Redis Cluster
Cache, sessions, rate limits, distributed locks for calendar holds

Object Storage (S3)
Photos, videos, documents. Served through the CDN

DynamoDB
Chat messages, notifications, high-write-volume data

5 SEARCH

Indexer (CDC)
Debezium streams DB changes via Kafka to the index, keeping search fresh in seconds

OpenSearch Cluster
Listings index: full-text, geo, filters, availability, ranking. Sharded + replicated

all services run on the platform

6 DEPLOYMENT & PLATFORM

Kubernetes (EKS)
Multi-AZ clusters, autoscaling, blue/green & canary rollouts

CI/CD (GitOps)
GitHub Actions: build, test, scan. ArgoCD: deploy to cluster

Infrastructure as Code
Terraform for VPC, DBs, queues. Separate dev / staging / prod

Observability
Prometheus + Grafana, ELK logs, OpenTelemetry tracing, Sentry

Security & Resilience
IAM, KMS, Secrets Manager, backups, multi-region DR

Illustrative reference architecture (AWS shown; design is cloud-agnostic).

Component Summary & Key Design Decisions

Layer	What it does	Technology (example)	Scale strategy
Frontend	Guest search and booking, host management, mobile apps, admin tools.	Next.js, React, React Native, TypeScript	SSR + CDN caching, code splitting, image optimization
Edge	Routes traffic, blocks attacks, serves static content close to users.	Route 53, CloudFront, AWS WAF, API Gateway, ALB	Global CDN, rate limiting, geo routing
Backend	Business logic split into independent services by domain.	Java / Node.js / Go, REST + gRPC, Kafka	Horizontal autoscaling, circuit breakers, async events
Storage / Database	Transactional data, cache, media, and high-volume chat data.	PostgreSQL, Redis, S3, DynamoDB	Read replicas, caching, partitioning, multi-AZ failover
Search	Fast listing discovery by location, dates, price, and amenities.	OpenSearch, Debezium CDC	Sharded index, replicas, near-real-time indexing
Deployment	Builds, ships, and monitors everything safely.	EKS, GitHub Actions, ArgoCD, Terraform, Grafana	Canary releases, IaC, multi-region disaster recovery

Key design decisions

- **No double bookings:** availability holds use Redis locks, and the final booking is committed in a PostgreSQL transaction.
- **Search stays fast:** listings are copied into OpenSearch through change-data-capture, so search load never hits the main database.
- **Loose coupling:** services talk through Kafka events (booking created, payment captured), so one failure does not cascade.
- **Right database for each job:** relational for money and bookings, key-value for chat, object storage for photos.
- **High availability:** multi-AZ deployment, automated failover, backups, and a multi-region disaster-recovery plan.
- **Safe releases:** GitOps with canary and blue/green rollouts; everything defined in Terraform and Git.
- **Security:** WAF at the edge, JWT auth, encryption in transit and at rest, PCI-compliant payment provider (no card data stored).